Intelligent Path-Finding Visualizer

(Search Algorithms)

Siva Sankar Reddy Veluri
40162043

Gundavarapu Mounika Satya Vani
40306499

1 Introduction

The process of finding a path through obstacles from a starting point to a target Goal point is one of the most fundamental problems with artificial intelligence systems. The problem exists in various practical situations which include robotics navigation, GPS routing, video games and logistics systems.

In this project, we built an interactive desktop application that visualizes how four different search algorithms solve the pathfinding problem on a grid. Users can design their own maze by placing walls, choosing a start and goal cell, and can visualize each algorithm exploring the space in real time. After each run, the application displays performance metrics including nodes expanded, path length, and execution time, allowing direct comparison of the algorithms on the same maze.

2 Technology Stack and Design Decisions

2.1 Programming Language : Python

Python was chosen for its simplicity, readability, and rich library ecosystem. For a project focused on algorithm logic and visualization, Python allows the core ideas to be expressed clearly without getting distracted by low-level implementation details.

2.2 Visualization Library: Pygame

Pygame was selected as the rendering library for the following reasons:

- It provides a simple window and drawing API suitable for grid-based visualizations.
- The system supports real-time animation through its event loop and frame-by-frame drawing.
- The application needs only basic configuration because it operates as one complete Python file which requires no web server or browser to function.
- It handles mouse and keyboard inputs natively, making the interactive grid editor straightforward to implement.

We also considered libraries like **tkinter** which is simpler but is very slow for animation and also tried a web-based approach using HTML/JavaScript where no installation is required, but it is harder to precisely control the timing of the animation using this approach..

2.3 Grid Design

We selected a 20x20 cells grid as it offers a visual balance by having sufficient space to draw mazes and also to keep the grid window small enough to fit with a standard laptop screen. Within the grid, each cell can hold seven different states which helps distinguish during path visualization.

The states are: Start, Goal, Wall, Empty, Frontier, Visited, Final Path

2.4 System Architecture

We maintained the entire code in a single "main.py" file to keep it simple as we don't need to import any custom modules or setup configuration files or add extra dependencies other than pygame. Any user with Python 3.12 and pygame installed can run the project with just a single command.

3 Application Features

3.1 Interactive Grid Editor

Users can:

- Place a **start cell** (green) anywhere on the grid using the S key
- Place a **goal cell** (red) anywhere on the grid using the G key
- **Draw walls** (black) by clicking and dragging using the W key
- **Erase** any cell using the E key
- **Clear** the search visualization while keeping the maze layout
- **Reset** the entire grid to start fresh

3.2 Real-Time Visualization

Each algorithm runs step by step with a small delay between each step (20ms). During animation:

- **Blue** cells represent the frontier, the nodes that have been discovered but not yet fully processed
- **Teal** cells represent visited nodes, the nodes that the algorithm has fully processed
- **Yellow** cells show the final path once the goal is reached

This color distinction will be helpful to the users to visually understand the difference between algorithms. For example, they can easily understand how BFS expands evenly outward while DFS dives deep in one direction.

3.3 Performance Metrics

After each run, the panel below the grid displays:

- The algorithm name
- Number of nodes expanded (how much of the space was explored)
- Path length (number of steps in the solution)
- Execution time (in seconds)

4 ALGORITHMS

4.1 Breadth-First Search (BFS)

BFS examines the grid through its various layers one by one. It uses a FIFO (First In First Out) or a Queue system to handle its node processing. The system processes all nodes at distance 1 from the start node before it proceeds to the nodes at distance 2 and continues this pattern. Due to this, the BFS algorithm achieves the shortest path in an unweighted grid.

- **Data structure:** Queue (deque)
- **Completeness:** Complete
- **Optimality:** Optimal. The system always determines the shortest route from the start node to the goal node.
- **Trade-off:** The system explores multiple nodes to achieve optimal pathfinding results

4.2 Depth-First Search (DFS)

DFS explores the grid by going as deep as possible along one branch before backtracking. The system processes nodes through the Stack approach which operates using Last In First Out (LIFO) methodology. The system finds a path which might not be the shortest or optimal.

- **Data structure:** Stack (list used as stack)
- **Completeness:** Complete

- **Optimality:** Not optimal as the paths given by the system are often too long.
- **Trade-off:** The system consumes memory at a lower rate than BFS, but creates lower quality path results

4.3 Greedy Best-First Search

The Greedy algorithm uses a priority queue which orders its elements according to a heuristic measurement that shows the estimated distance from each current node to the goal node using Manhattan distance as our chosen measurement method. The system selects its next node for expansion based on which node appears most similar to the target without measuring its previous distance traveled. Due to this, the execution of this algorithm is very fast, but it doesn't guarantee the shortest path.

- **Data structure:** Min-heap (priority queue)
- **Heuristic:** Manhattan distance i.e., Absolute value of row difference + absolute value of column difference
- **Completeness:** Complete
- **Optimality:** Not optimal as the system can be tricked into believing it found the best solution
- **Trade-off:** The system requires minimal node processing and is very fast, but might not give the shortest path.

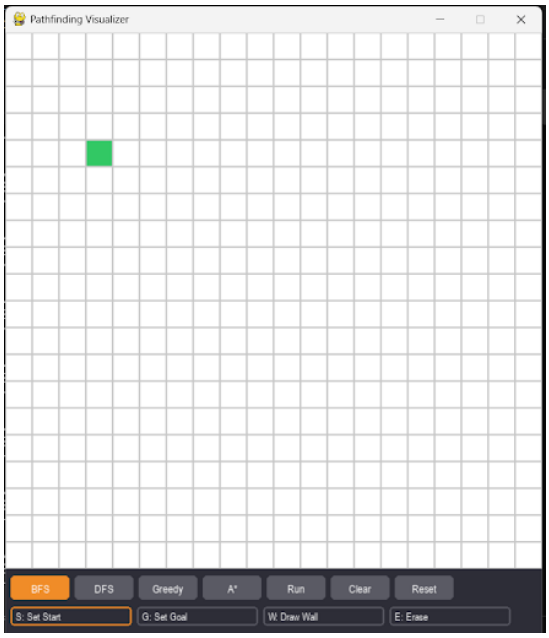
4.4 A* Search

A* algorithm combines the strengths of BFS and Greedy algorithms. The algorithm functions through a priority queue which determines path costs by using the equation f(n) = g(n) + h(n). Here, g(n) is the actual cost from the start node to the current node and h(n) is the heuristic estimate from the current node to the goal node. Hence, A* achieves the optimal path by examining fewer nodes compared to BFS because it evaluates both the cost components.

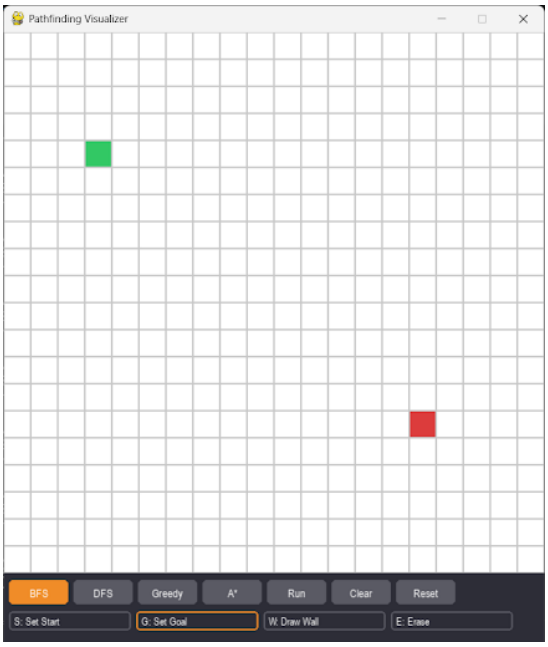
- **Data structure:** Min-heap (priority queue)
- **Heuristic:** Manhattan distance
- **Completeness:** Complete
- **Optimality:** This algorithm gives optimal results for admissible heuristics and Manhattan distance fulfills this.
- **Trade-off:** Finds optimal path, but a little more complex than Greedy algorithm.

5 Experimental Results

All four algorithms were tested on the same maze to enable direct comparison. The maze consisted of a 20×20 grid with a horizontal wall barrier containing multiple gaps, a start cell placed at the upper left area, and a goal cell placed in the lower right area.



(a) Start cell placed on the grid



(b) Goal cell placed on the grid

Figure 1: Initial setup of start and goal positions

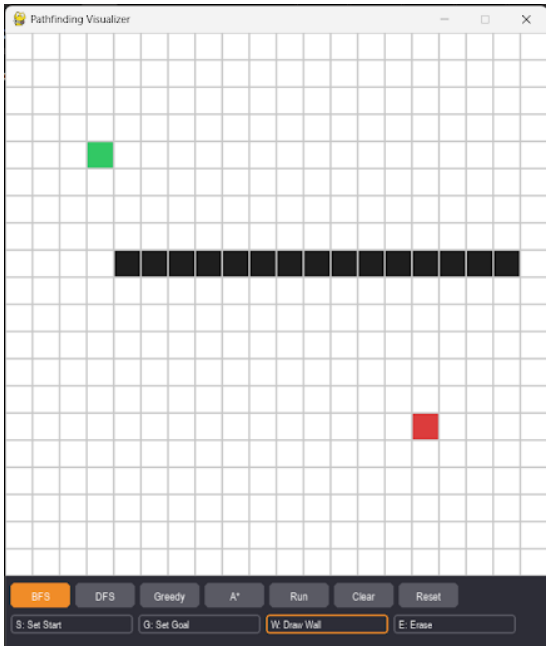
5.1 Maze Configuration

(See Figure 2(b) below - Initial maze with Start, Goal and walls with gaps, before any algorithm is run)

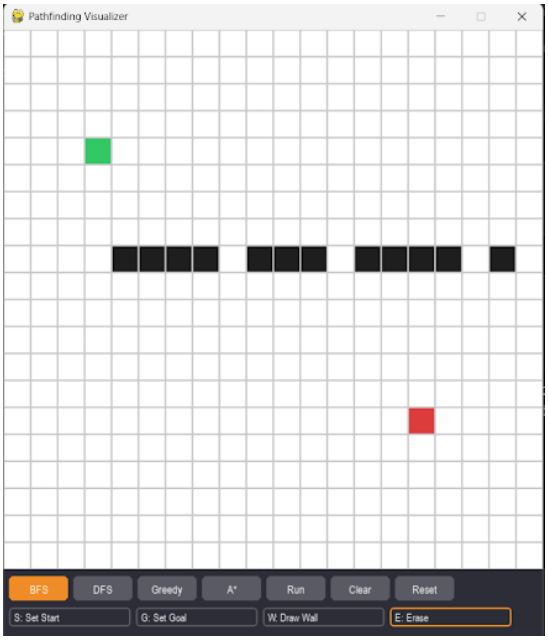
Start: upper-left area of the grid

Goal: lower-right area of the grid

Walls: a horizontal barrier with gaps, forcing the algorithms to navigate around or through



(a) Initial wall barrier added across the grid



(b) Gaps introduced in the wall to allow navigation paths

Figure 2: Maze construction process with barrier and gaps

5.2 Results Table

Algorithm	Nodes Expanded	Path Length	Execution Time
BFS	339	22	7.8753s
DFS	183	182	4.2344s
Greedy	31	26	0.7036s
A*	133	22	3.1125s

5.3 Analysis

BFS expanded the most nodes (339) but found the shortest possible path (length 22). Its systematic layer-by-layer exploration covered a large portion of the grid, which explains both the high node count and the longer execution time. The cells being filled with teal color evenly in all directions was clearly visible during animation.

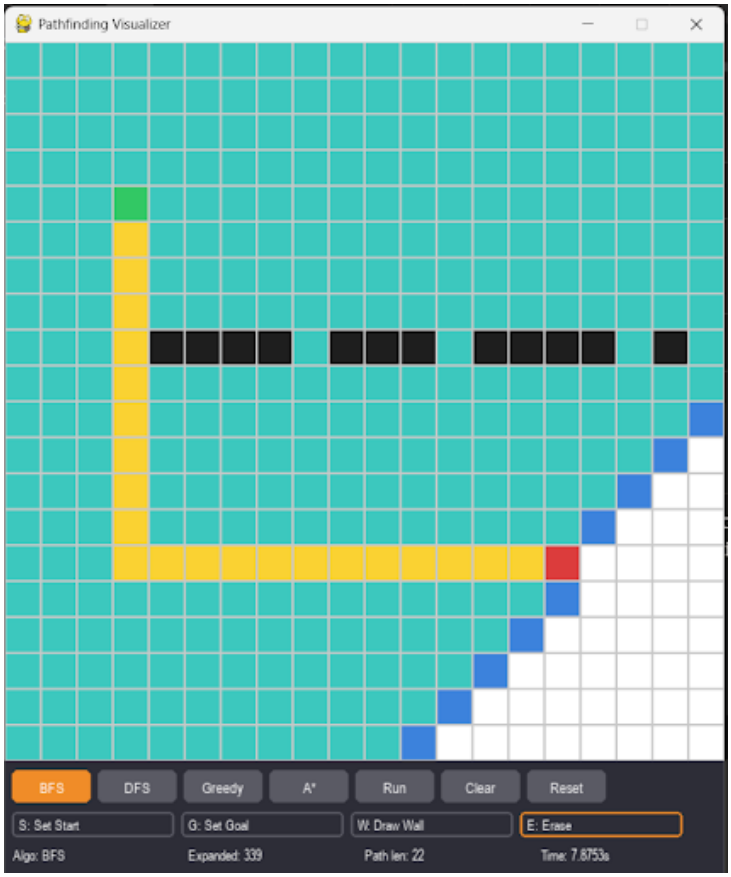


Figure 3: BFS exploration showing uniform expansion (teal) and optimal path (yellow)

DFS expanded fewer nodes (183) but produced an extremely long path (length 182, nearly 8 times longer than optimal). DFS went deep in one direction in a layer and then moved to the next layer, through most of the grid before reaching the goal. Though it found a path, it was far from being optimal. This is the core weakness of DFS, as it has no optimal path.

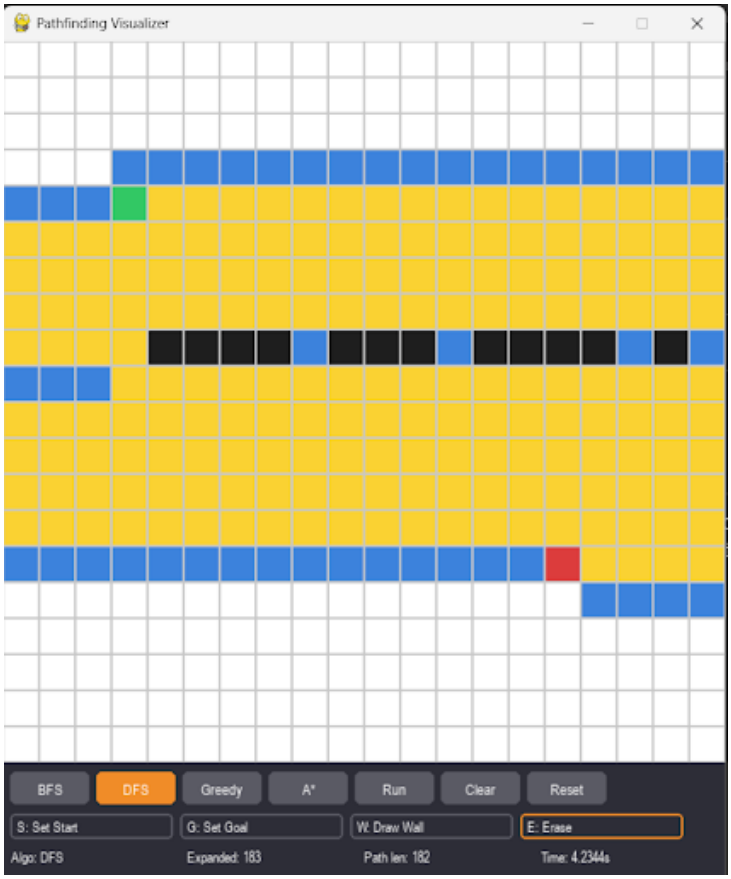


Figure 4: DFS exploration showing deep traversal and inefficient long path (yellow)

Greedy was the fastest by far, only 31 nodes expanded in 0.7036 seconds. The system followed a direct path to the target while using the Manhattan distance heuristic for navigation purposes. The path length produced by the system is 26, which exceeded the optimal path length of 22 by only a small margin. Greedy allows for detours which exceed actual travel distance because it only evaluates progress towards the goal node ignoring the actual distance travelled. The animation displayed a narrow exploration pattern which targeted a specific path while BFS showed a broad distribution of its search approach.

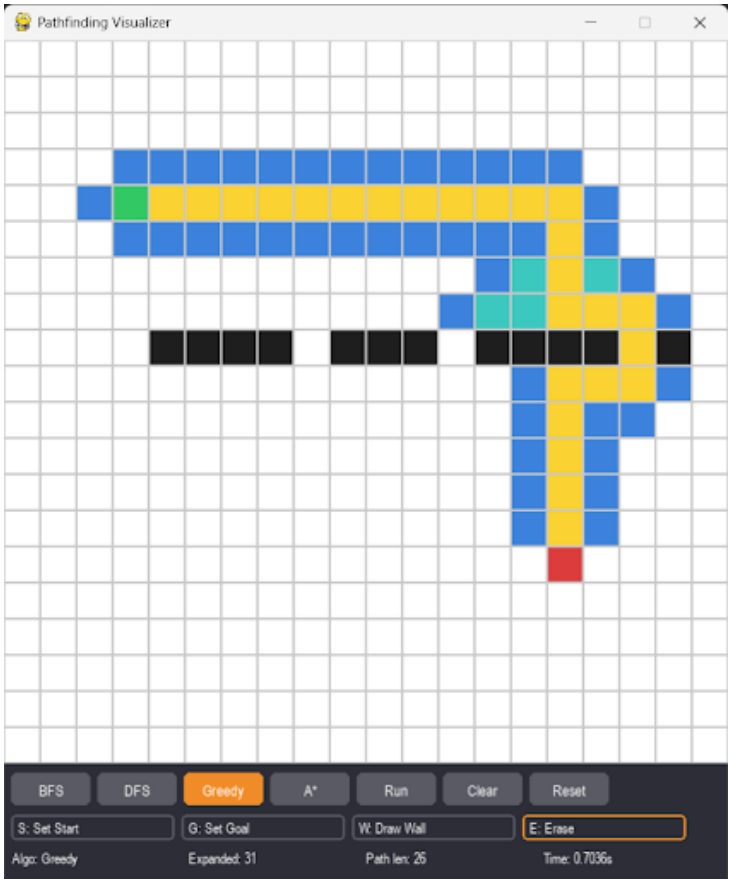


Figure 5: Greedy Best-First Search showing fast goal-directed exploration with slight deviation from optimal path

A* achieved the best overall balance among the 4 algorithms tested. The algorithm discovered the same optimal path length as BFS which measured 22 while it required the expansion of 133 nodes to complete the task. The system executed more efficiently through A* because it used less time for its computations, 3.1125 seconds, compared to BFS (7.8753 seconds). A* used actual path cost along with heuristic estimates to prevent time waste on unpromising paths while ensuring the detection of shortest paths..

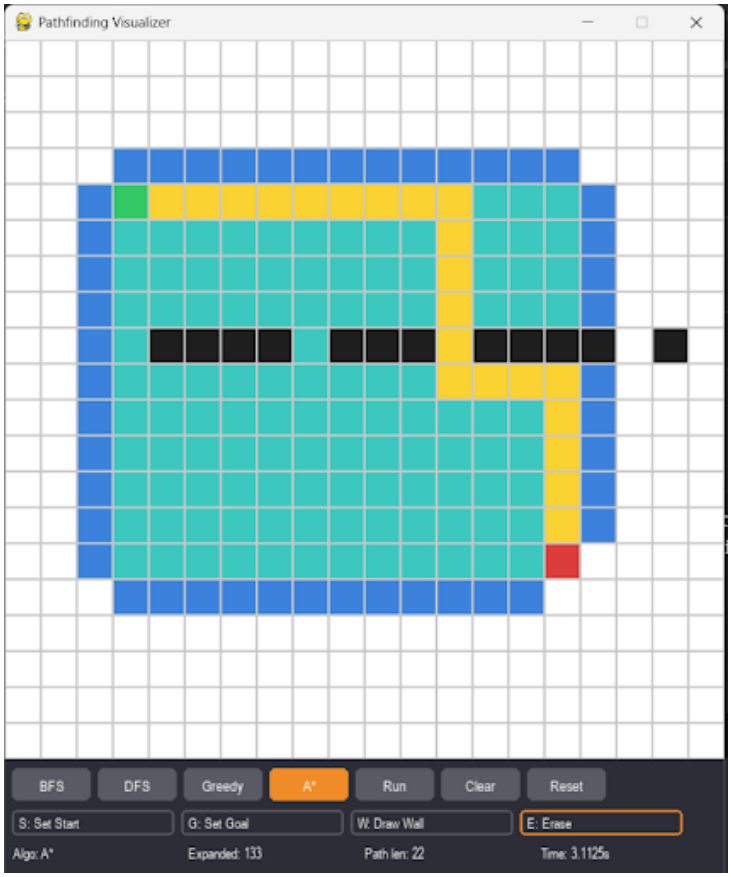


Figure 6: A* Search showing efficient exploration with optimal path (yellow) and reduced node expansion (teal)

5.4 Algorithm Comparison summary

Algorithm	Optimal	Complete	Speed	Node Expansion
BFS	Yes	Yes	Slow	Very High
DFS	No	Yes	Medium	Moderate
Greedy	No	Yes	Very Fast	Very Low
A*	Yes	Yes	Fast	Low

5.5 Effect of Maze Configuration on Results

The choice of start position, goal position, and wall layout directly affects all metrics for every algorithm. A different maze configuration would produce entirely different numbers. For example:

- Placing the goal in an open area with no walls would reduce expanded nodes for all algorithms.
- A maze with a single narrow corridor would cause BFS to expand many more nodes than Greedy.

- Placing the goal behind a wall with only one entry point would cause Greedy to fail to find the optimal path.

Our maze was designed to create a meaningful challenge, the wall barrier forced all algorithms to navigate around the obstacles, producing a clear distinction between their behaviors.

6 Challenges

6.1 Python Version Compatibility

The initial attempt to install pygame on Python 3.14 was failed because pygame 2.6.x does not yet support **distutils** module on Python 3.14 as it was removed. Hence we had to install Python 3.12 alongside 3.14 and use **py -3.12** to run the project. This highlights the importance of checking the compatibility of libraries before starting a project.

6.2 Animation vs. Algorithm State

The main technical challenge we faced was to integrate step-by-step animation into the algorithm loops without breaking the internal state of the algorithm. The solution was to call a **step()** function at the end of each iteration, which redraws the screen and processes pygame events (including the quit event) before the start of the next iteration.

6.3 Window Sizing

Fitting the grid, panel, buttons, mode indicators, and metrics all within a single window that fits on a standard laptop screen required several iterations. The final solution separates `GRID_SIZE` and `PANEL_H` as independent constants, making it easy to adjust without breaking the layout.

6.4 State Management Between Algorithm Runs

Switching between algorithms required reliably clearing the previous search state (visited, frontier, path cells) while preserving walls, start, and goal. Hence, we added a dedicated `clear_path()` function to handle this, and the start and goal cells are explicitly restored after clearing the grid to prevent them from being accidentally overwritten.

7 Conclusion

This project successfully implemented and visualized four fundamental AI search algorithms on an interactive grid. The key takeaways from the comparison are:

- **BFS** is the right choice when the shortest path is required and memory is not a constraint.
- **DFS** should generally not be used for pathfinding as it finds paths quickly but they are usually far from optimal.
- **Greedy** is the right choice when speed matters more than optimality and it is useful in real-time systems where a good enough path is acceptable.
- **A*** is the best general purpose pathfinding algorithm as it combines optimality with efficiency and is the standard choice in game development, robotics, and navigation systems.

The visualization makes these differences immediately intuitive. Watching BFS flood the grid versus watching Greedy rush toward the goal communicates the algorithmic difference more clearly than any textual description could.

8 References

- Pygame documentation: https://www.pygame.org/docs/
- Python documentation: https://docs.python.org/3.12/